

Embedded Machine Learning

Modul: Intelligente Eingebettete Systeme

Autoren: Fachbereich Informatik

Status: Studienheft v1.2

Akademisches Jahr 2024/25

1. Einleitung in Embedded Machine Learning

In den letzten Jahrzehnten hat sich das Paradigma der Datenverarbeitung massiv gewandelt. Während anfangs die zentrale Verarbeitung auf Großrechnern dominierte, führte der Weg über PCs hin zur Cloud. Heute erleben wir die nächste Evolution: **Edge AI** und **TinyML**.

1.1 Definition

Embedded Machine Learning (EML) ist die Disziplin, ML-Modelle so zu optimieren und zu implementieren, dass sie direkt auf Mikrocontrollern (MCUs) oder digitalen Signalprozessoren (DSPs) ausgeführt werden können. Das Hauptmerkmal ist, dass die Inferenz lokal erfolgt, ohne auf eine permanente Internetverbindung angewiesen zu sein.

Kernaussage: Embedded ML bringt die Intelligenz zum Sensor, anstatt die Sensordaten zur Intelligenz (Cloud) zu schicken.

1.2 Abgrenzung: Cloud vs. Edge

Merkmal	Cloud ML	Embedded ML (Edge)
Rechenleistung	Nahezu unbegrenzt (GPU/TPU)	Sehr begrenzt (MHz Bereich)
Speicher	Terabytes	Kilobytes bis Megabytes
Latenz	Abhängig vom Netzwerk	Echtzeit / Deterministisch
Stromverbrauch	Hoch (Netzbetrieb)	Extrem gering (Batterie)

2. Hardware für Embedded ML

Die Wahl der richtigen Hardware ist entscheidend für den Erfolg einer TinyML-Anwendung. Da wir uns im Bereich der Watt-Bruchteile bewegen, ist Effizienz wichtiger als rohe Gewalt.

2.1 Mikrocontroller (MCUs)

Die am weitesten verbreitete Plattform sind die ARM Cortex-M Serien (M0+, M4, M7, M33). Diese Prozessoren bieten Instruktionserweiterungen wie CMSIS-DSP, die mathematische Operationen für neuronale Netze beschleunigen.

2.2 Spezialisierte Beschleuniger (NPU's)

Moderne Chips integrieren zunehmend kleine Neural Processing Units (NPUs). Diese sind darauf spezialisiert, Matrix-Vektor-Multiplikationen mit minimalem Energieaufwand durchzuführen. Beispiele sind die Ethos-U Serie von ARM oder dedizierte KI-Chips wie der Syntiant NDP.

Exkurs: Das ESP32-S3 System

Der ESP32-S3 ist ein beliebtes Modul für Prototypen, da er Vektor-Instruktionen besitzt, die speziell die Beschleunigung von KI-Workloads unterstützen. Er bietet ein hervorragendes Verhältnis zwischen Kosten, Konnektivität und Rechenleistung.

3. Der TinyML-Entwicklungszyklus

Der Workflow bei Embedded ML unterscheidet sich signifikant vom klassischen ML-Training auf Servern.

3.1 Datenerfassung und Pre-Processing

Da Sensordaten auf eingebetteten Systemen oft verrauscht sind, ist eine robuste Vorverarbeitung (z.B. FFT für Audiosignale oder Filter für Beschleunigungssensoren) essenziell. Oft wird dieser Teil bereits in Hardware oder hocheffizientem C-Code umgesetzt.

3.2 Training und Evaluierung

Das Training findet weiterhin in der Cloud oder auf leistungsstarken Workstations statt (unter Verwendung von Frameworks wie TensorFlow oder PyTorch). Die Evaluierung muss jedoch bereits die Zielhardware simulieren.

3.3 Die Konvertierungsphase

Nach dem Training wird das Modell in ein Format wie **TensorFlow Lite for Microcontrollers** konvertiert. Hierbei wird das Modell "eingefroren" und für den Read-Only-Speicher optimiert.

Wichtig: Ein Modell, das in der Cloud 99% Genauigkeit erreicht, kann auf dem Mikrocontroller versagen, wenn die numerische Stabilität durch Quantisierung verloren geht.

4. Modelloptimierung

Ohne Optimierung passen moderne neuronale Netze nicht in den Flash-Speicher eines Mikrocontrollers. Die drei Säulen der Optimierung sind:

4.1 Quantisierung

Standardmodelle verwenden 32-Bit Fließkommazahlen (float32). Bei der Quantisierung werden diese auf 8-Bit Ganzzahlen (int8) reduziert. Dies spart 75% Speicherplatz und ermöglicht die Nutzung von Hardware-Einheiten, die Ganzzahlen deutlich schneller verarbeiten.

4.2 Pruning (Strukturelles Beschneiden)

Viele Gewichte in einem neuronalen Netz haben Werte nahe Null und tragen kaum zum Ergebnis bei. Beim Pruning werden diese Verbindungen entfernt. Man unterscheidet zwischen:

- **Unstructured Pruning:** Einzelne Gewichte werden auf Null gesetzt.
- **Structured Pruning:** Ganze Neuronen oder Filter werden entfernt.

4.3 Knowledge Distillation

Ein kleineres "Studenten-Modell" lernt, das Verhalten eines großen, vortrainierten "Lehrer-Modells" zu imitieren. Das Ziel ist es, die Entscheidungslogik des großen Modells in einer Architektur mit deutlich weniger Parametern abzubilden.

5. Praxisanwendungen

Embedded ML findet sich heute in fast jedem modernen Haushaltsgerät und Industriesensor.

5.1 Predictive Maintenance

Sensoren an Industriemotoren analysieren Vibrationsmuster. Ein lokales ML-Modell erkennt Abweichungen, die auf einen bevorstehenden Lagerschaden hindeuten, bevor dieser eintritt.

5.2 Audio-Analyse

- **Keyword Spotting:** Aufweckwörter wie "Alexa".
- **Geräuscherkennung:** Erkennung von Glasbruch in Sicherheitssystemen.

5.3 Computer Vision am Edge

Obwohl Bildverarbeitung extrem rechenintensiv ist, ermöglichen spezialisierte Architekturen wie **MobileNet** oder **Person Detectors** die Anwesenheitserkennung auf Sensoren, die nur mit einer Knopfzelle betrieben werden.

Zukunftstrend: On-Device Learning

Aktuelle Forschung beschäftigt sich damit, Modelle nicht nur auszuführen, sondern direkt auf dem Gerät an den Nutzer anzupassen (z.B. Personalisierung von Gestensteuerung).

6. Zusammenfassung und Ausblick

In diesem Kapitel haben wir gelernt:

- Dass Embedded ML die Lücke zwischen Sensorik und Intelligenz schließt.
- Warum Energieeffizienz wichtiger ist als maximale Genauigkeit.
- Wie Quantisierung und Pruning Modelle "schrumpfen".
- Dass die Hardware-Wahl (Cortex-M, NPUs) den Software-Stack bestimmt.

Selbstkontrollfragen

1. Erklären Sie den Unterschied zwischen Training und Inferenz im Kontext von TinyML.
2. Warum ist die Reduzierung von float32 auf int8 vorteilhaft für MCUs ohne FPU?
3. Nennen Sie drei Vorteile der lokalen Datenverarbeitung gegenüber der Cloud-Lösung.
4. Was versteht man unter dem Begriff "CMSIS-NN"?

Ende des Kapitels 2.3 - Weiterführende Literatur finden Sie im Anhang des Studienhefts.